

**INSTITUTO FEDERAL**  
Goiás

Bacharelado em Sistemas de Informação  
Disciplina: Programação Orientada a Objetos I

# Associação de Classes Manipulando Exceções

Prof. Ms. Dory Gonzaga Rodrigues  
Goiânia - GO

## Manipulação de Exceções

O uso das exceções em um sistema é de extrema importância, pois ajuda a detectar e tratar possíveis erros que possam acontecer.

### O que é uma Exceção ?

Exceções são problemas que ocorrem e provocam a interrupção do fluxo normal do programa.

Para tratar essas situações, temos os manipuladores de exceções.

A ideia é passar o fluxo de controle, uma vez interrompido por uma exceção, para um código que será responsável por manipular e procurar uma alternativa a um código que não pode ser executado por completo.

Exemplo: um método tenta abrir um arquivo em disco e ler algumas informações. Por algum motivo, o método não conseguiu abrir esse arquivo e não pode ser executado, assim o programa para a execução,

Temos que ter uma saída e a saída é um manipulador de exceção.

### Problemática

Chamamos aqui de Problema do Saldo Negativo.

```
public class Conta {  
    private String numero;  
    private double saldo;  
    ...  
    public void debitar(double valor) {  
        saldo = saldo - valor;  
    }  
}
```

Problema:

O método debitar() sempre fará o débito do valor informado, independente de ter saldo ou não !!!!

Nesta situação, o sistema deve tratar o problema de alguma forma.

### Problemática

Chamamos aqui de Problema do Saldo Negativo.

#### Solução 1

```
public class Conta {  
    private String numero;  
    private double saldo;  
    ...  
    public void debitar(double valor) {  
        if (valor <= saldo) {  
            saldo = saldo - valor;  
        }  
    }  
}
```

A solução possui um erro de lógica.

O débito não ocorrerá, caso o valor a ser debitado seja maior que o saldo, e ninguém é avisado !!!

O sistema não gera qualquer tipo de aviso, e para o usuário o débito ocorreu normalmente

### Problemática

Chamamos aqui de Problema do Saldo Negativo.

### Solução 2

```
public class Conta {  
    private String numero;  
    private double saldo;  
    ...  
    public void debitar(double valor) {  
        if (valor <= saldo)  
            saldo = saldo - valor;  
        else  
            System.out.print("Saldo Insuficiente !!!");  
    }  
}
```

Uma mensagem no console é gerada avisando que o saldo é insuficiente !  
Vários objetos Conta estão sendo manipulados ao mesmo tempo pelo sistema !!!  
Qual objeto gerou a mensagem de erro?

### Problemática

Chamamos aqui de Problema do Saldo Negativo.

### Solução 3

```
public class Conta {  
    private String numero;  
    private double saldo;  
    ...  
    public boolean debitar(double valor) {  
        boolean r = false;  
        if (valor <= saldo) {  
            saldo = saldo - valor;  
            r = true;  
        }  
        return r;  
    }  
}
```

O método retorna true caso o débito ocorra e false caso contrário.

O problema dessa solução é o fato de que a saída do método não deveria ser utilizada para esse propósito, já que se o método realizasse algum processamento que precisasse retornar algum valor, não conseguiríamos mais utilizar essa solução.

Dessa forma o método fica impedido de utilizar algum valor como saída que não seja true ou false

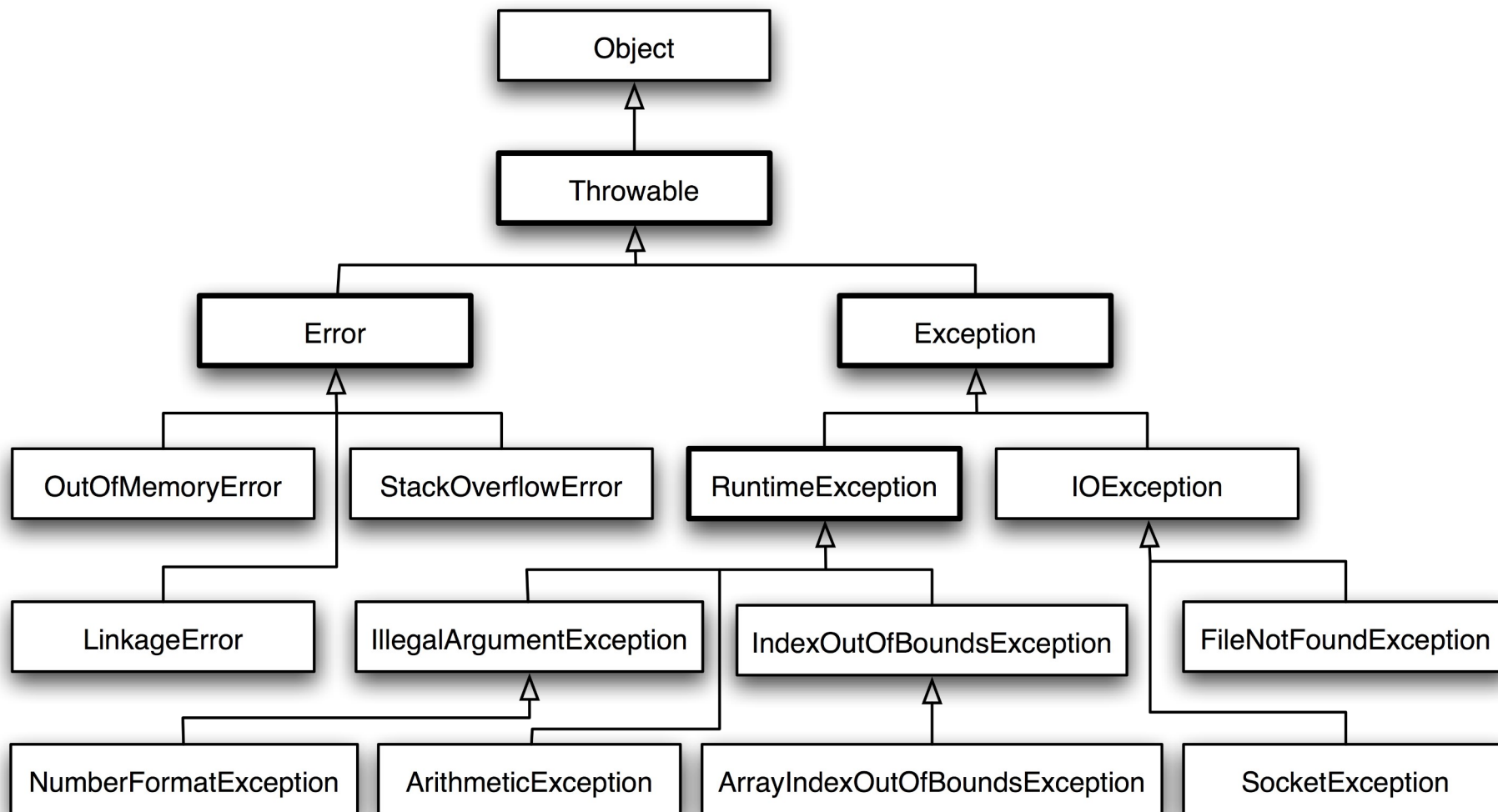
### Exceções

Usando exceções:

- Não usamos código para tratar os problemas
- Teremos objetos Java comuns que herdam de uma superclasse que representa estas exceções.
- Essas classes Java são subclasses da classe Exception.
- A classe Exception pertence ao pacote java.lang.
- Na criação de exceções, definiremos subclasses de Exception onde será possível oferecer informações extras sobre a falha e distinguir entre os vários tipos de falhas geradas.



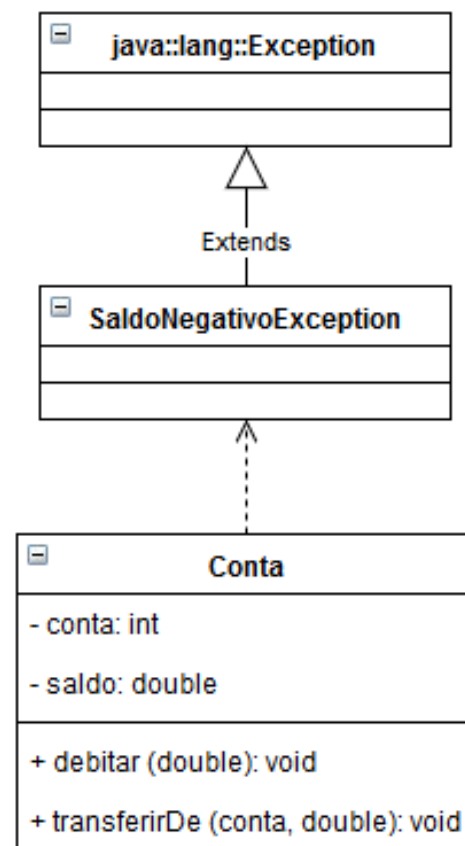
## Manipulação de Exceções



### Manipulação de Exceções

O uso das exceções em um sistema é de extrema importância, pois ajuda a detectar e tratar possíveis erros que possam acontecer.

O uso de exceções garante o estado consistente da aplicação.



## Palavra Reservada: Throw e Throws

TO THROW significa: “LANÇAR, ARREMESSAR”.

- Na assinatura do método debitar(), é colocado no final (depois da declaração de parâmetros) a indicação de que o método debitar() “lança” a exceção: “SaldoNegativoException”.
- Com essa indicação, sabe-se de antemão que em algum lugar no método debitar() a exceção SaldoNegativoException será “lançada”.
- Em Java é obrigatório a indicação junto à assinatura do método de quais exceções esse método irá lançar. Caso isso não seja feito, é gerado erro de compilação.
- Como a exceção é um objeto Java comum, deve ser criado um objeto da classe SaldoNegativoException e lançado através da palavra reservada throw.

### Exceções

#### Solução 4 - Com uso de Exceções

#### Na classe SaldoNegativoException

```
public class SaldoNegativoException extends Exception {  
    public SaldoNegativoException(String num) {  
        super("Saldo Insuficiente em: " + num);  
    }  
}
```

#### Na classe Conta

```
public class Conta {  
    ...  
    public void debitar(double valor) throws SaldoNegativoException {  
        if (valor <= saldo)  
            saldo = saldo - valor;  
        else {  
            throw new SaldoNegativoException(this.numero);  
        }  
    }  
}
```

A classe que representa a exceção **SaldoNegativoException**.

Observação:

1) o construtor da classe chama o construtor da classe `Exception` passando a mensagem de erro.

2) recebe em seu construtor um parâmetro do tipo `String` que representa o número da conta que gerou a exceção. Esse valor será passado no momento do lançamento da exceção.

### Palavra Reservada: Throw e Throws

#### LANÇAMENTO DIRETO

- O método que declara uma exceção e o faz internamente com o uso do throw.

```
public class Conta {  
    ...  
    public void debitar(double valor) throws SaldoNegativoException {  
        if (valor <= saldo)  
            saldo = saldo - valor;  
        else {  
            throw new SaldoNegativoException(this.numero);  
        }  
    }  
}
```

#### LANÇAMENTO INDIRETO

- O método que declara o lançamento não é o mesmo que o faz, mas sim algum método interno chamado por ele.

```
public class Conta {  
    ...  
    public void transferirDe(Conta c, double v) throws SaldoNegativoException {  
        this.debitar(v);  
        c.creditar(v);  
    }  
}
```

### Tratamento de Exceções

- Até o momento apenas fizemos o lançamento de falhas de forma direta ou indireta !
- Tratamento é quando damos um destino às falhas encontradas, tentar uma solução alternativa, avisar sobre o erro gerado, contornando o problema de forma elegante e simples.
- O lançamento de exceções existe quando o desenvolvedor não quer que a exceção seja tratada e sim lançada para que um método acima deste (método que o chama) a trate.
- Em algum ponto a exceção deve ser capturada e tratada, e para o tratamento de exceções utilizamos os blocos `try-catch`.

### Bloco de comando: try-catch

- “TO TRY” do inglês “TENTAR” e
- “TO CATCH” do inglês “CAPTURAR”.
- A ideia por trás dos blocos try-catch é tentar executar uma chamada de método, e caso a exceção por ele lançada ocorra, capturá-la para dar algum destino. Isto é, tentar tratar o problema depois de ocorrido.

```
public static void main(String[] args) {  
  
    Conta conta = new Conta(...);  
  
    try {  
        ...  
        conta.debitar(90.00);  
        ...  
    } catch (SaldoNegativoException e) {  
        System.out.println(e.getMessage());  
    }  
    /* outros blocos catch */  
}
```

### Bloco de comando: finally

Existe também, opcionalmente, o bloco finally

```
public static void main(String[] args) {  
  
    Conta conta = new Conta(...);  
  
    try {  
        ...  
        conta.debitar(90.00);  
        ...  
    } catch (SaldoNegativoException e) {  
        System.out.println(e.getMessage());  
    }  
    finally {  
        /* liberando recursos */  
    }  
}
```

- É utilizado para fechar algum recurso que foi aberto durante a execução do try. Neste caso, é garantido que o recurso será fechado mesmo que ocorram exceções durante a execução do bloco try.

- O bloco finally é opcional, mas caso ele exista, é um bloco que será executado de qualquer forma.

- Quando um try é executado sem nenhum problema, o controle é passado para o bloco finally. Caso o try lance alguma exceção, o catch correspondente é chamado e logo em seguida o finally.



### Cuidado com o uso dos blocos try-catch e finally

Abaixo segue um resumo com as precauções que devem ser tomadas, sendo algumas já apontadas:

- Só deve existir um único bloco try para vários catches;
- Um bloco try deve ser seguido de pelo menos um catch ou um finally;
- Método que anuncia o lançamento de uma exceção não pode capturar a mesma exceção dentro de um catch

```
public void teste() throws SaldoNegativoException {  
    try {  
        debitar(100);  
    }  
    catch (SaldoNegativoException sne) { }  
}
```

- Se um método lança uma exceção, essa mesma exceção deve ser anunciada por ele;
- Exceções que herdam de RuntimeException não são anunciadas;
- Se um método qualquer chama outro método que lança uma exceção, esse método deve tratá-la ou relançar a mesma exceção (anunciando).